DBMS Lecture 11 — Normalization (Study Notes)

Oct 25, 2026 · @Utkarsh

1. Overview

Normalization is the technique used to remove **redundancy** (repeated data) from a database. It is one of the most important, interview-specific topics in DBMS. The lecture builds up to it in this order:

1. **Functional dependency (FD)** — the mathematical language used to describe which attribute determines which.
2. **Types of FD** — trivial and non-trivial.
3. **Armstrong's axioms** — rules to derive new FDs from given ones (reflexive, augmentation, transitivity), with solved questions.
4. **Why normalization?** — redundancy causes three anomalies (insertion, deletion, updation), inconsistency and a bigger, slower DB.
5. **What normalization does** — decompose a table into smaller tables until each one follows the Single Responsibility Principle.
6. **Normal forms** — 1NF → 2NF → 3NF → BCNF, each stricter than the previous.
7. **Advantages** of normalization.

The lecturer's typed notes open with a one-line definition worth remembering: **Normalisation is a step towards DB optimisation.**

> Key idea to carry through the whole lecture: whenever you design a DB, you want **no redundancy**. Every normal form removes one more kind of dependency that causes redundancy.

2. Functional Dependency (FD)

A functional dependency **A → B** (read "A determines B") means: if you are given the value of A, you can determine the value of B. It is just a mathematical way of writing *which attribute depends on which*.

Definition (from notes): It's a relationship between the primary key attribute (usually) of the relation to that of the other attribute of the relation.

Intuition — it feels like a primary key

If you know the primary key, you can fetch the rest of the row. An FD captures exactly this idea, but more generally: "if you tell me the value of A, I can find the value of B."

Take a table with attributes A, B, C, D:

A	B	C	D
1	2	1	5
2	4	5	6

- If A → B holds and I give you A = 1, you search the table and return B = 2.
- If I give you A = 2, you return B = 4.
- Similarly A → C can also hold.

Determinant and Dependent

In X → Y:

Side	Name	Meaning
Left (X)	**Determinant**	It has the ability to determine the other value. You supply its value.
Right (Y)	**Dependent**	Its value depends on the left side; it is found once the left side is known.

Important: the left side A can be a **single attribute or a set of attributes**. Just as two attributes together can form a (composite) primary key, a set like {Emp_id, Name} can be a determinant.

Example — Employee table

Emp_id	Emp_name	dept

If Emp_id is the primary key, the FDs are:

- **Emp_id → Emp_name** — give me an Emp_id and I can find the specific employee's name.
- **Emp_id → dept** — give me an Emp_id and I can search the table and find the department.

Here Emp_id is the determinant; Emp_name and dept are dependents. "FD" is the common short form of functional dependency.

3. Types of Functional Dependency

An FD A → B is **trivial** when B is a subset of A, and **non-trivial** when B is not a subset of A.

3.1 Trivial FD

Rule: A → B is trivial if **B ⊆ A** (the dependent is a subset of the determinant).

"Trivial" means ordinary or obvious — it is bound to be true; it is simple mathematics.

Example: In the Employee table, suppose {Emp_id, Name} is a candidate key. Then:

- **{Emp_id, Name} → Emp_id** is trivial. If you know Emp_id and Name, you obviously know Emp_id. The right side (Emp_id) is a subset of the left side ({Emp_id, Name}).

More examples: A → A and B → B are also trivial, because A ⊆ A and B ⊆ B. If I give you the value of Emp_id, you can of course "find" Emp_id.

3.2 Non-trivial FD

Rule: A → B is non-trivial if **B is not a subset of A** (written B ⊄ A). The notes also state it as **A ∩ B = NULL**.

Example 1: Add an Address column to the Employee table, with {Emp_id, Name} as the candidate key.

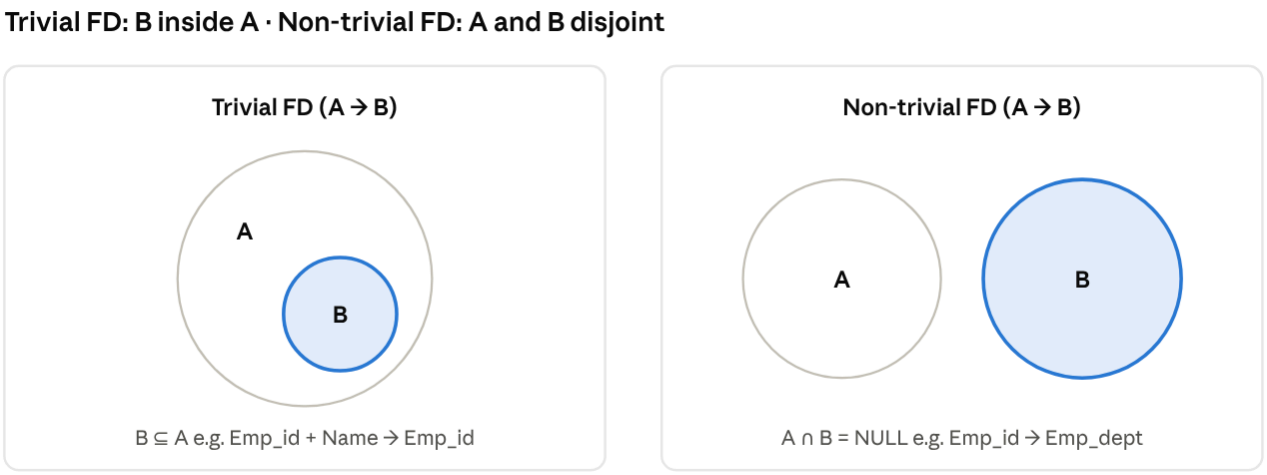
- **{Emp_id, Name} → {Emp_address}** — given both Emp_id and Name, you can find the address.
- Determinant = {Emp_id, Name}; dependent = {Emp_address}.
- Is {Emp_address} ⊆ {Emp_id, Name}? **No.** So this is a non-trivial FD.

Example 2: Emp_id → Emp_dept is non-trivial, because department is not a subset of ID.

Quick recap

Type	Condition	Example
Trivial	B ⊆ A	{Emp_id, Name} → Emp_id; A → A
Non-trivial	B ⊄ A (A ∩ B = NULL)	{Emp_id, Name} → Emp_address; Emp_id → Emp_dept

In general: you give the value of an attribute (ideally a candidate key) and if you can find another attribute from it, that is an FD. Whether it is trivial or non-trivial depends only on whether the right side is already contained in the left side.



Venn diagrams · trivial vs non-trivial FD

In the trivial case B lies wholly inside A; in the non-trivial case the two sets do not overlap at all.

4. Rules of FD — Armstrong's Axioms

Armstrong's axioms are three rules that let you derive new, valid FDs from the FDs you are given. You must know all three: **Reflexive, Augmentation, Transitivity**.

Rule	Formal statement	One-line intuition
Reflexive	If B ⊆ A, then A → B	A set always determines any part of itself
Augmentation	If A → B, then AX → BX	Adding the same attributes to both sides keeps the FD valid
Transitivity	If A → B and B → C, then A → C	Determination chains through

4.1 Reflexive rule

Notes: If A is a set of attributes and B is a subset of A, then A → B holds. In symbols: if A ⊇ B, then A → B.

(The typed notes on screen say "B is a subset of 'X'" — this is a typo; it means B is a subset of A.)

Example:

- X = {a, b, c, d, e}
- Y = {a, b, c}, so Y is a subset of X.
- Therefore **X → Y**.

Why it is true: Y is already part of X. If you know the value of X, you already know the values of Y. This is very similar to a trivial FD.

4.2 Augmentation rule

Notes: If B can be determined from A, then adding an attribute to this functional dependency won't change anything. If A → B holds, then AX → BX holds too, where X is a set of attributes.

Example: Relation R(X, Y, Z) with the given FD X → Y.

- Augment both sides with Z: **XZ → YZ** is also valid.
- You can add a third, fourth, or any set of attributes, as long as you add it to **both** sides.

Why it is true: X gives you Y. Z on the left gives you Z on the right (reflexively). So together XZ gives you YZ.

4.3 Transitivity rule

Notes: If A determines B and B determines C, we can say that A determines C. If A → B and B → C, then A → C.

This is the same transitivity you studied in mathematics. A, B and C here can each be a set of attributes.

```
A → B,  B → C   ⟹   A → C
```

5. Solved Questions on Armstrong's Axioms

Both questions use the same relation and the same given FDs:

```
R(A, B, C, D, E)

Given FDs:
  A  → B
  A  → C
  CD → E
  B  → D
  E  → A
```

Method: to check whether an FD is legal, try to *derive* it from the given FDs using only the axioms. If you can, it is valid; if you cannot, it is illegal.

Question 1 — Which of these FDs is illegal?

(a) BC → CD — Valid ✔

1. Given: B → D.
2. Augment both sides with C: BC → DC, i.e. **BC → CD**.

(b) EC → AC — Valid ✔

1. Given: E → A.
2. Augment both sides with C: **EC → AC**.

(c) BD → CD — Illegal ✘

1. This looks like augmentation with D. Remove D from both sides and you would need **B → C**.
2. Is B → C given, or derivable? No such FD is given. B only gives D (B → D), and D alone gives nothing new.
3. So BD → CD cannot be derived. It is an **illegal FD**.

Trick: when both sides share an attribute, strip it off and ask whether the remaining FD holds. That tells you whether augmentation can produce it.

Question 2 — Prove that CD → AC

Step 1 — get CD → A (transitivity)

- Given: CD → E
- Given: E → A
- By transitivity: **CD → A**

Step 2 — get CD → C (reflexive / trivial)

- C ⊆ CD, so by the reflexive rule: **CD → C**. This is a trivial FD — the dependent is a subset of the determinant.

Step 3 — combine

- CD determines A and CD determines C, so CD determines both together: **CD → AC ✔ Valid**.

```
CD → E  and  E → A    ⟹  CD → A    (transitivity)
C ⊆ CD                ⟹  CD → C    (reflexive)
CD → A  and  CD → C   ⟹  CD → AC
```

The lesson: combine the axioms. Reflexive gives you the "free" parts of the left side; transitivity carries you through chains; augmentation lets you pad both sides.

6. Why Normalization? Redundancy and Anomalies

We normalize **to avoid redundancy in the DB — i.e. not to store redundant data.** Normalization is the optimization technique that turns a DB with redundant information into a well-designed DB where redundancy is minimal or nearly zero.

What happens if we have redundant data?

Redundant data introduces **three anomalies** (anomaly = abnormality):

1. **Insertion anomaly**
2. **Deletion anomaly**
3. **Updation anomaly** (also called **modification anomaly**)

The running example — a badly designed Student table

Someone built this table without thinking about redundancy. It mixes student data and branch data in one place:

id	name	age	Branch_code	Branch_name	Branch_HOD
1	A	18	1	CS	X
2	B	19	1	CS	X
3	C	18	1	CS	X
4	D	21	2	ECE	Y
5	E	20	2	ECE	Y
6	F	19	3	M.E	Z

Notice CS / X is repeated three times and ECE / Y twice. (If you had thought carefully at the ER-model stage, you would already have made separate Student and Branch tables. Assume nobody did.)

6.1 Insertion anomaly

Definition: When certain data (attribute) cannot be inserted into the DB without the presence of other data.

Example 1 — a new student with no branch yet. Student 7, name G, age 21 has enrolled but is still in counselling and has not chosen a branch.

id	name	age	Branch_code	Branch_name	Branch_HOD
7	G	21	NULL	NULL	NULL

You either cannot insert the student at all, or you insert NULLs and come back later to update them. That is double work.

Example 2 — a new department with no students. The university introduces IT, Branch_code = 4. But no student has enrolled in IT yet. In this merged table you cannot add IT without a student — or you must fill the student columns with NULLs.

Root cause: the existence of a student and the existence of a branch are **independent**, but the table has merged them. So one cannot be inserted without the other.

6.2 Deletion anomaly

Definition: The situation where the deletion of data results in the unintended loss of some other important data.

Example: Student 6 (F) is the only Mechanical (M.E) student. F passes out, so we delete row 6.

- Deleting that row also deletes the only record that M.E exists, with its code 3 and HOD Z.
- The DB now looks as if the university does not teach Mechanical Engineering at all.
- Important information was lost just because a student was removed.

With separate Student and Branch tables, this would not happen.

6.3 Updation (modification) anomaly

Definition: When an update of a single data value requires multiple rows of data to be updated.

Example: The CS HOD changes from X to Q.

- Every row of every CS student must be updated: rows 1, 2 and 3 each change X → Q.
- With more CS students, more rows must change.
- One logical change becomes many physical changes.

Consequence — data inconsistency: Due to updating in many places, data inconsistency may arise if one forgets to update the data at all the intended places. If one or two rows are missed, CS now has two different HODs in the DB.

6.4 Summary of problems caused by redundancy

1. The three anomalies (insertion, deletion, updation).
2. **Data inconsistency.**
3. **Bigger DB size** — the same branch information is stored again and again. A separate Department table would need only one row per department (e.g. 5 departments = 5 rows).
4. **Slower queries** — searching a bigger DB takes longer.

7. Fixing It — What Normalization Does

Normalization **decomposes a table into multiple smaller tables** until each table follows the **Single Responsibility Principle (SRP)**.

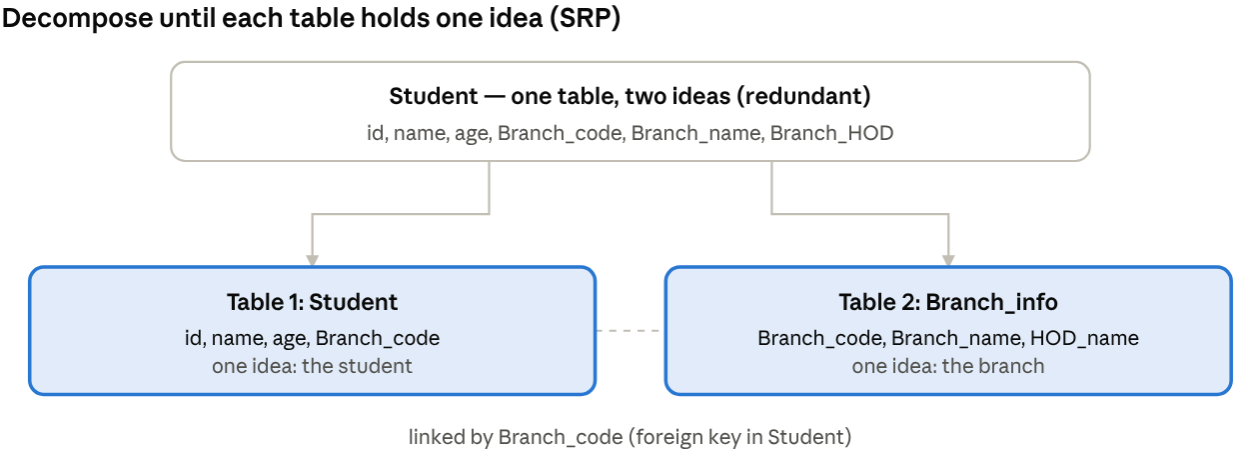
7.1 Fixing the Student table

Even before learning the normal forms, the natural fix is to split the table in two:

Table 1 — Student: id, name, age, Branch_code

Table 2 — Branch_info: Branch_code, Branch_name, HOD_name

Branch_code	Branch_name	HOD_name
1	CS	X
2	ECE	Y
3	ME	Z
4	IT	Q



decomposing the Student table · 1 table into 2

Student keeps only Branch_code; all branch details live once in Branch_info.

Now check each anomaly again:

- **Insertion:** IT (code 4, HOD Q) is added to Branch_info even though no student has enrolled in IT yet. A new student can be added independently too.
- **Deletion:** deleting the last M.E student removes only the student row; M.E stays in Branch_info.
- **Updation:** if the CS HOD changes X → Q, only **one row** in Branch_info changes.
- The DB is also easier to view and search, smaller, and free of those inconsistencies.

So **no insertion, deletion or updation anomaly** remains.

7.2 Single Responsibility Principle (SRP)

What we do in normalization: Table → decompose → into multiple tables, **until SRP is achieved**.

SRP: each table in the DB should have **only one job** — it should represent a single idea. The original Student table stored student information *and* branch information, so it had two responsibilities. We keep decomposing until every table represents exactly one idea.

7.3 What is normalization? (formal notes)

1. Normalisation is used to minimise the redundancy from a relation. It is also used to eliminate undesirable characteristics like insertion, update and deletion anomalies.
2. Normalisation divides the composite attributes into individual attributes, OR a larger table into smaller ones, and links them using relationships.
3. The normal form is used to reduce redundancy from the database table.

7.4 The normal forms ahead

We move step by step from one normal form to the next, adding stricter restrictions each time and optimizing the DB further:

1NF → 2NF → 3NF → BCNF

Each normal form requires the previous one to already hold.

8. First Normal Form (1NF)

A table is in 1NF when **every cell holds a single, atomic value** — no multi-valued attributes. Every table should at least be in 1NF.

Rules (notes):

1. Every relation cell must have an atomic value.
2. The relation must not have multi-valued attributes.

Atomic means the value cannot be divided further. You have seen this earlier in the course: if you store a contact number, one cell should hold exactly one number.

Example — Employee phone numbers

Not in 1NF (employee 2 has two phone numbers in one cell):

id	name	Phone
1	A	88
2	B	12, 99

Converted to 1NF (one value per cell; the row is repeated for each phone):

id	name	Phone
1	A	88
2	B	12
2	B	99

Important observation — 1NF still allows redundancy

In the converted table, "2, B" is repeated. 1NF does **not** care about repetition; it only demands that each attribute's values are atomic.

In practice, you avoid this repetition at design time: when converting an ER diagram to tables, a **multi-valued attribute gets its own separate table** (as learned in the ER-to-relational lectures). Your mind is already trained to apply normalization at that stage. But theoretically, 1NF on its own only produces the table above.

9. Second Normal Form (2NF)

A table is in 2NF when it is in 1NF **and has no partial dependency**: every non-prime attribute depends on the **whole** primary key, not on just a part of it.

Rules (notes):

1. Relation must be in 1NF.
2. There should not be any partial dependency.
   a. All non-prime attributes must be fully dependent on the PK.
   b. A non-prime attribute cannot depend on a part of the PK.

Key terms

Term	Meaning
Prime attribute	An attribute that is part of the primary (candidate) key
Non-prime attribute	Any attribute that is not part of the key
Full dependency	A non-prime attribute is determined only by the entire key
Partial dependency	A non-prime attribute is determined by just a part of a composite key

Partial dependency can only happen when the primary key is **composite** (made of 2+ attributes). Even with a composite key, each non-prime attribute must depend on all of its attributes together.

9.1 Abstract example — R(A, B, C, D)

- Primary key = **{A, B}** (A and B together).
- Prime attributes: **A, B**. Non-prime attributes: **C, D**.
- What *should* hold: **AB → C** and **AB → D** (the whole key determines the rest).
- Given FD: **B → C**.

B → C is a **partial dependency**: the dependent C is determined by **B alone**, which is only a part of the determinant (the key AB).

Why is this a problem? (lecturer's reasoning) The key is the combination AB. The lecturer argues that only the combination is guaranteed to be meaningful, so one part of it (say B) could be NULL in some row. If B is NULL, B → C cannot hold — NULL cannot determine any value. So a non-prime attribute must be determined by the whole key, never half of it.

> Note for interviews: in standard SQL no column of a primary key is allowed to be NULL. The more widely quoted reason partial dependency is bad is **redundancy** — C is repeated for every row with the same B, which brings back the insertion, deletion and updation anomalies. The fix is the same either way.

Conversion to 2NF — decompose:

```
R(A, B, C, D)      PK = {A, B},   FD: B → C  (partial)

        ⟹   R1(A, B, D)   PK = {A, B}    AB → D   (full dependency)
            R2(B, C)      PK = B         B  → C   (full dependency)
```

In each new table, the non-prime attribute depends on the whole key of *that* table.

9.2 Real-life example — Student_project table

The name itself hints that two pieces of information (student and project) are being stored together.

Student_ID	Project_ID	Student_Name	Project_Name
S89	P09	Ovia	Geo
S76	P07	Jacob	Cluster
S56	P03	Ava	IOT
S92	P05	Alex	Cloud

- Is it in 1NF? **Yes** — all values are atomic.
- Primary key = **{Student_ID, Project_ID}**.
- Prime: Student_ID, Project_ID. Non-prime: Student_Name, Project_Name.
- FDs:
  - **Student_ID → Student_Name** — determined by a part of the key → **partial dependency**.

- **Project_ID → Project_Name** — determined by a part of the key → **partial dependency**.

The whole key was supposed to determine the non-prime attributes, but each one depends on only one half. So the table is **not in 2NF**.

Conversion to 2NF — break the table:

Student table (PK = Student_ID; Project_ID kept as a foreign key):

Student_ID	Project_ID	Student_Name
S89	P09	Ovia
S76	P07	Jacob
S56	P03	Ava
S92	P05	Alex

Project table (PK = Project_ID):

Project_ID	Project_Name
P09	Geo
P07	Cluster
P03	IOT
P05	Cloud

Now:

- **Student_ID → Student_Name** is a **full** dependency, since Student_ID is the whole primary key of the Student table.
- **Project_ID → Project_Name** is a **full** dependency, since Project_ID is the whole primary key of the Project table.

Why is Project_ID still in the Student table? To keep the **link** between the two tables. Project_ID is a **foreign key** in Student, referring to the primary key of Project. (This works because, in this data, each student has exactly one project.)

2NF in one line: remove every partial dependency by moving the attributes that depend on part of the key into their own table.

10. Third Normal Form (3NF)

A table is in 3NF when it is in 2NF **and has no transitive dependency**: a non-prime attribute must not determine another non-prime attribute.

Rules (notes):

1. Relation must be in 2NF.
2. No transitive dependency exists.
    a. A non-prime attribute should not find a non-prime attribute.

We are moving step by step: each normal form adds a stricter restriction and optimizes the DB further.

10.1 Example — R(A, B, C)

- Primary key = **A** (so A is prime; B and C are non-prime).
- FDs: **A → B** and **B → C**.

By transitivity, A → B and B → C already give A → C. So why is B → C sitting there separately? That separate path through a non-prime attribute is the problem.

Step 1 — is it in 2NF?

- Values are atomic (assume 1NF holds).
- The key A is a single attribute, so nothing can depend on "part" of it. B and C are fully dependent on the primary key.
- So yes, it **is in 2NF**.

Step 2 — look at the data to see the redundancy that transitivity causes:

A (PK)	B	C
a	1	x
b	1	x
c	1	x
d	2	y
e	2	y
f	3	z
g	3	z

- A → B: give any unique A and you find its B.

- B → C also holds: wherever B = 1, C = x; wherever B = 2, C = y; wherever B = 3, C = z.
- So a **non-prime attribute (B) is finding another non-prime attribute (C)**.

(While explaining, the lecturer briefly says "a non-prime attribute is finding a prime attribute" — that is a slip of the tongue, which he corrects. C is non-prime here; the 3NF rule is about **non-prime → non-prime**.)

Why is this wrong? We created the primary key precisely so that *it* determines everything. Here a non-prime attribute is independently determining other data. The visible result is **redundancy**: the pairs (1, x), (2, y), (3, z) are repeated again and again.

B → C is called a transitive dependency.

10.2 Conversion to 3NF

Decompose so the transitive FD gets its own table:

```
R(A, B, C)    PK = A,   FDs: A → B, B → C (transitive)

      ⟹   R1(A, B)   PK = A
          R2(B, C)   PK = B
```

R1:

A (PK)	B
a	1
b	1
c	1
d	2
e	2
f	3
g	3

R2:

B (PK)	C
1	x
2	y

B (PK)	C
3	z

R1 still needs one row per A (A is the determinant there, so those rows are necessary). But the B → C information now needs only **three rows** in R2. The repeated x, y, z values are gone — the redundancy is removed. B becomes a prime attribute (PK) in R2, so there it is the key determining a non-prime attribute, which is allowed.

3NF in one line: no non-prime attribute may determine another non-prime attribute; move such a chain into its own table.

11. Boyce-Codd Normal Form (BCNF)

BCNF (**Boyce-Codd Normal Form**) is a **stronger version of 3NF**: the table must be in 3NF, and for every FD A → B, **A must be a super key**. Equivalently, no attribute — prime or non-prime — may be used to derive a prime attribute unless it is a super key.

Rules (notes):

1. Relation must be in 3NF.
2. For every FD A → B, A must be a super key.
    a. We must not derive a prime attribute from any prime or non-prime attribute.

Why another normal form after 3NF?

- 2NF removed: part of key → non-prime.
- 3NF removed: non-prime → non-prime.
- But a case remains where the **right-hand side is a prime attribute**: something determines a part of the key itself. If a prime attribute is already part of the key, why is it being determined with the help of some other attribute? That is still a problem, and BCNF removes it.

(Revise **super key** from the earlier keys lecture: a set of attributes that can uniquely identify every row.)

11.1 Example — Student, Subject, Professor

Stud_ID	Subject	Professor
101	Java	PJ
101	CPP	PC

Stud_ID	Subject	Professor
102	Java	PJ2
103	C#	PC#
104	Java	PJ

Constraints (business rules):

1. One student can enroll in multiple subjects (101 takes Java and C++).
2. For each subject, a professor is assigned to the student.
3. Multiple professors can teach a single subject (Java is taught by PJ and PJ2).
4. **One professor can teach only one subject.**

These constraints are deliberately chosen to illustrate BCNF.

Finding the key: Java has two professors, so Subject alone can't tell you who teaches a student. You need both: e.g. Stud_ID 102 + Java → PJ2. So the primary key is **{Stud_ID, Subject}** — together they determine all other columns.

- Prime attributes: **Stud_ID, Subject**.
- Non-prime attribute: **Professor**.

FDs:

1. **{Stud_ID, Subject} → Professor** — the whole key determines the non-prime attribute. Fine.
2. **Professor → Subject** — because each professor teaches only one subject, knowing the professor tells you the subject.

Check the earlier normal forms:

- 1NF: yes, all values are atomic.
- 2NF: yes, no partial dependency (Professor depends on the whole key).
- 3NF: yes, no non-prime → non-prime dependency (Subject is prime, not non-prime).

Why it is NOT in BCNF: in FD 2, a **non-prime attribute (Professor) is determining a prime attribute (Subject)**. We are going in reverse — trying to find a part of the key from a non-key attribute. Professor is not a super key (it is not even a key), so the BCNF rule "A must be a super key" fails.

The lecturer's shortcut: in BCNF, the right-hand side of an FD should never be a prime attribute derived from a non-super-key. Neither a prime nor a non-prime attribute may be used to find a prime attribute.

11.2 Conversion to BCNF

Give each professor an ID and split the table so the professor-to-subject fact is stored once, keyed by the professor:

Student table:

Stud_ID	P_id
101	1
101	2
…	…

Professor table (PK = P_id):

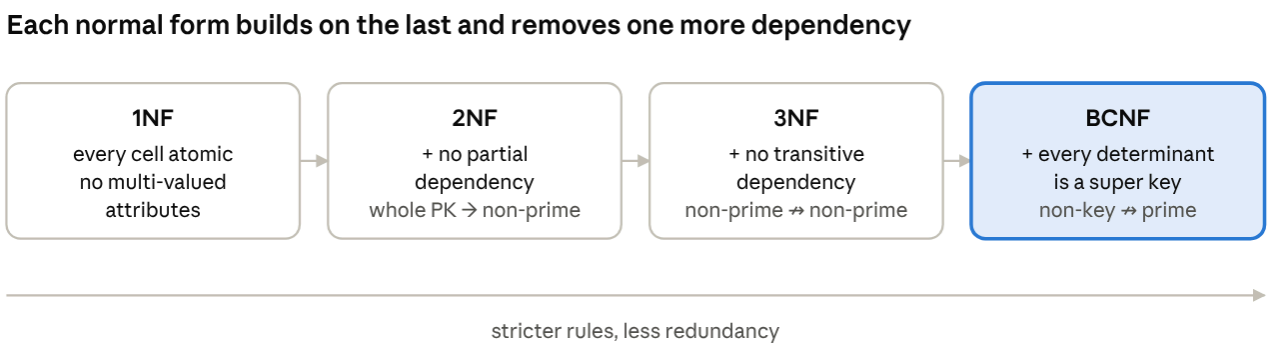
P_id	Professor	Subject
1	PJ	Java
2	PC	CPP
…	…	…

- In the Professor table, **P_id is the prime attribute (key)** and it determines the non-prime attributes Professor and Subject.
- PJ teaches Java; PJ2 would also teach Java, but with a **different P_id**.
- No non-prime attribute is determining P_id; every determinant is now a key.

So the condition holds and the design is in **BCNF**.

12. Normal Forms Compared, Advantages and Revision

You start at 1NF and move to 2NF, then 3NF, then BCNF; each form requires the previous one and forbids one more kind of dependency.



normal-form ladder · 1NF to BCNF

Read left to right: each box keeps every rule of the box before it and adds one more.

12.1 Side-by-side (for an FD α → β)

Normal form	Requires	Forbidden dependency	Fix
1NF	—	Multi-valued / non-atomic cells	One value per cell
2NF	1NF	β (non-prime) depending on a **part** of the PK (partial dependency)	Move the partially dependent attributes to a table keyed by that part
3NF	2NF	Non-prime → non-prime (transitive dependency)	Move the chain (B → C) into its own table keyed by B
BCNF	3NF	Any α → β where α is not a super key (e.g. non-prime → prime)	Split so every determinant is a key

12.2 Advantages of normalization

1. Normalisation helps to **minimise data redundancy**.
2. **Greater overall database organisation** — a well-structured DB.
3. **Data consistency** is maintained in the DB.

The chain of benefits explained in the lecture: less redundancy → no insertion/deletion/updation anomalies → no data inconsistency → a smaller DB that takes less space → queries run faster and more optimized.

12.3 Interview revision checklist

- **FD A → B:** given A, you can determine B. A = determinant (single attribute or a set); B = dependent.

- **Trivial FD:** $B \subseteq A$ (e.g. $A \rightarrow A$, {Emp_id, Name} → Emp_id). **Non-trivial:** $B \not\subseteq A$, $A \cap B$ = NULL.
- **Armstrong's axioms:** Reflexive ($B \subseteq A \Rightarrow A \rightarrow B$), Augmentation ($A \rightarrow B \Rightarrow AX \rightarrow BX$), Transitivity ($A \rightarrow B, B \rightarrow C \Rightarrow A \rightarrow C$).
- **Legal-FD check trick:** strip common attributes from both sides and see if the remaining FD is given or derivable ($BD \rightarrow CD$ needs $B \rightarrow C$, which is not given ⇒ illegal).
- **Why normalize:** redundancy ⇒ insertion, deletion, updation anomalies + inconsistency + bigger, slower DB.
- **Insertion anomaly:** can't insert one thing without another (new student without branch; new IT branch without students).
- **Deletion anomaly:** deleting one thing loses another (last M.E student deleted ⇒ M.E branch disappears).
- **Updation anomaly:** one change needs many rows updated (CS HOD X → Q in every CS row); missing a row ⇒ inconsistency.
- **What normalization does:** decompose tables until SRP — each table represents one idea.
- **1NF:** atomic values, no multi-valued attributes (repetition is still allowed).
- **2NF:** 1NF + no partial dependency. Only possible with a composite key.
- **3NF:** 2NF + no transitive dependency (non-prime must not determine non-prime).
- **BCNF:** 3NF + for every $A \rightarrow B$, A is a super key; a stronger version of 3NF.
- **Linking after decomposition:** the split tables are joined through a foreign key (e.g. Project_ID in Student, Branch_code in Student).